← Retour index Python Niveau II

Python Niveau II — Plan de cours détaillé

Durée : 2 jours (14h) — progression continue, exercices à difficulté croissante.

Jour 1 — Objet et décorateurs

Créneau	Progression	Contenu	Exercice guidé	Critère de réussite
09:00–10:30	Base	Refonte d'un script en objets : responsabilités, attributs, méthodes, dataclasses.	Modéliser une entité métier (Commande, Client, Produit).	Chaque classe a une responsabilité claire.
10:45–12:30	Intermédiaire	Composition vs héritage, interfaces implicites, contrats de méthodes, introduction au data model Python.	Refactor d'un héritage excessif en composition.	Couplage réduit, meilleure lisibilité.
13:30–15:00	Intermédiaire+	Décorateurs : `wraps`, paramètres, logging, validation.	Créer `@validate_input` et `@log_duration`.	Décorateurs réutilisables et testables.
15:15–17:00	Consolidation	Atelier de synthèse J1 : architecture orientée objet + décorateurs.	Mini-moteur de traitement (entrée/sortie + règles métier).	Code cohérent, sans duplication majeure.

Jour 2 — Coroutines et async/await

Créneau	Progression	Contenu	Exercice guidé	Critère de réussite

09:00–10:30	Base	Event loop, coroutine, `await`, `create_task`, `gather`.	Paralléliser 20 appels I/O simulés.	Temps d'exécution significativement réduit.
10:45–12:30	Intermédiaire	Gestion des erreurs async : timeout, annulation, retry, backoff.	Rendre tolérant un lot d'appels API instables.	Aucun crash global, erreurs tracées proprement.
13:30–15:00	Intermédiaire+	Limiter la concurrence (`Semaphore`), éviter surcharge et throttling.	Respecter une limite de 5 appels concurrents.	Débit stable, pas d'explosion d'erreurs.
15:15–17:00	Avancé	Projet final : service asynchrone complet orienté objet.	Assembler modèle, décorateurs, orchestration async.	Livrable exécutable + quiz + recommandations.

Bloc théorique transverse (niveau II)

- **Méthodes magiques** : implémenter correctement `__repr__`, `__len__`, `__iter__`, `__eq__` selon les besoins métier.
- **Cycle de vie d'objet** : rôle de `__init__`, `__new__`, immutabilité et invariants.
- **MRO et héritage multiple** : lecture et impact sur la maintenabilité.
- **Conception API Pythonique** : exceptions explicites, méthodes courtes, noms cohérents, docstrings ciblées.
- **Async en production** : backpressure, cancellation, retry idempotent, observabilité minimale.

Repères de code du niveau II

```
import functools

def positive_amount(fn):
```

```
    @functools.wraps(fn)
    def wrapper(amount, *args, **kwargs):
        if amount <= 0:
            raise ValueError("amount doit être positif")
        return fn(amount, *args, **kwargs)
    return wrapper
```

```
import asyncio

sem = asyncio.Semaphore(5)

async def safe_call(call):
    async with sem:
        return await asyncio.wait_for(call(), timeout=2)
```

```
class Money:
    def __init__(self, amount: float, currency: str = "EUR") -> None:
        self.amount = amount
        self.currency = currency

    def __repr__(self) -> str:
        return f"Money(amount={self.amount}, currency='{self.currency}')"

    def __add__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("devise incompatible")
        return Money(self.amount + other.amount, self.currency)
```

Stack : Python 3.11+, pytest, asyncio, VS Code.

Détails techniques approfondis (niveau II)

Conception objet

- **Critère 1** : une classe doit pouvoir être résumée en une phrase métier.
- **Critère 2** : les invariants sont validés au plus tôt (constructeur / factory).
- **Critère 3** : les dépendances techniques sont injectées, jamais cachées dans la classe métier.
- **Critère 4** : les méthodes publiques exposent un contrat stable (types + exceptions attendues).

Décorateurs en production

- Préférer plusieurs décorateurs simples à un décorateur « tout-en-un » difficile à tester.
- Versionner les politiques transverses (timeout, retry, logging) pour suivre leur évolution.
- Mesurer le surcoût de chaque couche décorative sur les chemins chauds.
- Exiger un test nominal + un test d'erreur pour chaque décorateur critique.

Asynchrone robuste

- Définir des limites explicites: concurrence max, timeout, taille de lot, tentatives max.
- Utiliser des files (`Queue`) pour lisser la charge entre producteurs et consommateurs.
- Tracer les événements de cancellation pour éviter les tâches fantômes.
- Documenter les opérations idempotentes avant d'ajouter du retry automatique.

Glossaire opérationnel

- **Invariants métier** : règles non négociables qui définissent la validité d'un objet.
- **Concern transverse** : besoin récurrent hors logique métier (log, auth, tracing, retry).
- **Throughput** : volume d'opérations traitées par unité de temps.
- **Latency** : temps de réponse d'une opération individuelle.
- **Backoff exponentiel** : stratégie de retry augmentant progressivement le délai.

Définitions techniques complémentaires (niveau II)

- **Polymorphisme** : capacité à utiliser des objets différents via une même interface.
- **Duck typing** : compatibilité basée sur les comportements disponibles, pas sur l'héritage explicite.
- **Wrapper** : fonction intermédiaire ajoutant un comportement autour d'une fonction cible.
- **Event loop starvation** : blocage de l'ordonnanceur async par une opération synchrone longue.
- **Task cancellation** : arrêt explicite d'une coroutine via une exception `CancelledError`.
- **Structured logging** : logs à structure stable (champs clés/valeurs) facilitant la recherche.
- **Contrat d'API interne** : convention partagée sur paramètres, retour, erreurs et effets de bord.

Approfondissement technique (niveau II)

Conception orientée objet maintenable

- Définir des ports (interfaces/Protocol) pour isoler le domaine des dépendances techniques.
- Réserver l'héritage à une relation métier claire; privilégier la composition dans les autres cas.

- Encapsuler les transitions d'état dans des méthodes explicites pour préserver les invariants.

Décorateurs et observabilité

- Injecter un identifiant de corrélation dans les logs pour suivre une opération de bout en bout.
- Mesurer latence et taux d'échec par décorateur afin de détecter les régressions de comportement.
- Tester séparément le code métier et la couche décorative pour accélérer le diagnostic.

Asynchrone en conditions réelles

- Rendre explicites les limites opérationnelles: timeout, concurrence max, tentatives, taille de lot.
- Gérer la fermeture propre des tâches pour éviter les fuites lors d'un arrêt applicatif.
- Utiliser `TaskGroup` pour centraliser la gestion d'échec d'un ensemble de coroutines liées.

```
import asyncio

async def worker(name: str, queue: asyncio.Queue) -> None:
    while True:
        payload = await queue.get()
        try:
            await asyncio.sleep(0.05)  # I/O simulée
            print(f"{name}: {payload}")
        finally:
            queue.task_done()
```